

“Moro’s Problem”: Finding grammar in the brain

Robert C. Berwick

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

Abstract. One of the main challenges of language science is to draw inferences about *internal* computations from *external* observations—in recent terminology, to deduce what is *I-language* from *E-language*. Since grammars, human or otherwise, are presumptively part of brain-*internal* computational systems, their activity is typically not directly observable, for example, in terms of the sentences or “language” that comprise *external* behavior. This gap is part of a long history of a tension between reductionist and emergentist approaches in the analysis of complex systems, extensively and explicitly analyzed, perhaps most famously explicated by David Marr in terms of his notion of “three levels of explanation.” More recently, in the context of speech and language, David Poeppel has dubbed this the “mapping problem,” and during the past two decades there have been many explicit attempts by neuroscientists and cognitive/computational linguists to address this issue. Here we examine one specific computational way in which Marr’s levels and Poeppel’s mapping problem might be concretely realized: how “grammars” engaged as language parsers can be obscured from direct observation, even given (hypothetical) direct observation of the human brain’s neural activity at the individual neuronal level in known language areas. This thought experiment suggests that the challenge

of how linguistic computations are actually implemented in the brain might be even more difficult than generally has been thought. It amounts to an updating of David Marr's methodological program that might well also be called "Moro's Problem," in light of Andrea Moro's efforts in a series of research papers over the past several decades that also aim to bridge between direct brain recordings and what we know about human language syntax.

Introduction

What makes inference about the human language faculty difficult? At heart, many have noted that problems arise because the link between 'internal' mind/brain neuronal computation and ultimately 'external' language behavior amounts to a gap separated by many "bridging assumptions"—a difficulty recently and cogently reviewed by Poeppel 2012, dubbed by him "the mapping problem." Poeppel's analysis amounts to a detailed unpacking of Marr's (1982) familiar heuristic for partitioning any computation into three levels: problem statement, algorithm, and implementation, where the mapping from problem to algorithm to implementation is in general many-to-one in both directions, and so possibly difficult to invert.

Many other researchers over the past several decades have worked out specific examples of Marr and Poeppel's analyses of these inferential difficulties. While there is insufficient space here for a complete review of all this work, a few references serve to illustrate the wide range of such examples. For example, Prinz, Bucher & Marder 2004 demonstrated that even if neuronal connectivity were precisely known, in the case of a 3-cell neural network for modeling a simple motor pattern (crustacean pyloric rhythm), the target behavior could be realized via a huge range of distinct synaptic strengths and other open parameters. In this case, output behavior greatly underdetermines the network implementation. More generally, in a widely known publication, Jonas & Kording 2017 showed that for a more complex, but also exactly known computational system, namely a 6502 microprocessor, careful signal measurements like those widely

used by neuroscientists in model biological organisms were insufficient to “reverse engineer” the microprocessor functionality and wiring. In this case as well, strict reductionism does not seem to suffice.

Finally, among many efforts closer to what is analyzed here, human language processing, several have advanced particular “bridging assumption” models and noted the difficulties that arise. For instance, Hale et al. 2022 underscore Poeppel’s point about the long inference chain connecting brain to behavior by advancing a very particular parsing model and grammar along with specific computational cost assumptions. They attempt to map an online measure of processing load as a “linking hypothesis” all the way from parser and grammar assumptions to predicted brain activity while subjects read stories (e.g., *Alice in Wonderland*). In this case, the “implementation” is at the level of parser operations, rather than neuronal topology. Clearly, each one of the Hale et al. assumptions are tendentious and call for independent support—and that is part of their message. A similar, but simpler point is made in what follows, for a very different human parsing algorithm.

In short then, the challenge of balancing reductionism and emergent behavior has a long, storied history, stretching from at least the time of the Enlightenment to the more recent work cited above. This challenge has also been tackled by Andrea Moro in a creative series of experiments probing human grammars via brain imaging (Musso et al. 2003; Moro 2015; Moro 2016, among others). In what follows then, with some slight historical distortion because this volume is dedicated to Moro rather than to the many others who have addressed the same issue, we will call this particular difficulty “Moro’s Problem.” Moro’s Problem is one in a long line of “problem statements” in cognitive science, akin to “Plato’s Problem” (how we come to know so much about language and the world given the impoverished experience that infants presumptively receive).

Perhaps the most (in)famous *early* example of Moro’s Problem is that of Jacques de Vaucanson’s “Digesting Duck” (*Canard Digéshotur*), 1739. In the familiar account, during an age consumed by the distinction between machines and people, de Vaucanson constructed an automaton that apparently consumed food and water, quacked,

waddled, and defecated. To be sure, the duck was partly meant “just” a joke, but de Vaucanson was completely serious about his reductionism. The duck, along with his other automata “were philosophical experiments, attempts to discern what aspects of living creatures could be reproduced in machinery” (Riskin 2003:601). Pointedly, de Vaucanson *deliberately* constructed his duck to be transparent, so that one could see its inner workings: “his ‘Design [had been] rather to demonstrate the Manner of the Actions, than to shew a Machine’ (“L,” pp. 22-23)” (Riskin 2003:608).¹ In this way, de Vaucanson emphasized the gulf between “E” (external behavior) and “I” (internal computational system), and that reduction to machinery demanded some non-identity mapping. More to the point here, there can typically be many grammars that generate exactly the same external strings, a difficulty that we return to in what follows. Indeed, after the duck was lost, later authors had to guess at its workings, sometimes incorrectly, as indicated by the legend for the figure in Wikipedia’s entry on Vaucanson’s duck, “An American artist’s (mistaken) drawing of how the Digesting Duck may have worked.”²

Moro’s Problem: Can we find grammar in the brain?

To see how challenging Moro’s problem might be even in an imagined future, we might imagine a greatly advanced brain imaging science, much further along than the fMRI and related methods used by Moro and beyond our current reach, but still plausible, say in the next century, 2101. Here we show that even given such “science fiction” imaging methods, pinpointing grammar in the brain might prove difficult. The point is that the way that “grammars” are implemented as processing engines can be far from transparent, and we will illustrate this with a concrete example that seems to have gone largely unnoticed.

1. As Riskin notes, this was not entirely true; part of the construction was deliberately hidden and fraudulent.

2. See: https://en.wikipedia.org/wiki/Digesting_Duck#/media/File:Digesting_Duck.jpg.

In this “science fiction” scenario, let us suppose that advanced “optogenetics” has enabled us to determine what neurons are connected to what other neurons in exact detail, along with the precise signaling between them, down to the level of dynamical change in every synaptic connection along with all the associated neuronal topology. It is not a completely outlandish idea: current optogenetics technology uses genomics and light sensitive proteins such as microbial opsins to “read out” the changes in individual neurons (Boyden et al. 2005). This is in fact Boyden’s explicit research goal. Let us suppose, then, that this program is entirely successful, and that in the year 2101 we can look at the region that we presumptively know to be the location of the language processing associated with the “Basic Operation” of syntax. We can now picture some future descendant of Moro’s art who has persuaded a 22nd century neurosurgeon dealing with patients like those subjects in the here-and-now Moro’s ECOG studies, and who can carry out direct observation of the brain.

What would our future Moro be able to see? Suppose finally that instead of just a tangle of neurons, optogenetic neural analysis reveals the hardwired *ground truth* of human grammar, as depicted in the table in Figure 1. Here, along the top row of the table we have Neurons (or perhaps very small neuronal assemblies), numbered 0 through 13, along with an “input” to each labeled a through j . Each row indicates that a particular input along the left-hand column causes the Neurons along the corresponding column at the top to send an activation signal to a new Neuron, given by the number in the corresponding (row, column) cell. For example, if the input is a , then Neuron 0 sends an activation signal to Neuron 2; Neuron 7 also sends an activation signal to Neuron 2. Note that cells can also be empty, indicating that the row, columns of the corresponding neurons are not linked. We have been deliberately vague here about what we mean here by an “input,” which we will reveal in what follows. But here is the punchline: the matrix *is* the grammar.

Perhaps surprisingly, even given that we know that this (still abstract) wiring diagram represents the *ground truth* of how the linguistic grammar is “wired up” it is *still* very difficult—indeed essentially impossible—to figure out *what* is implemented and *where* the

grammar really is. This is of course no different at all in kind from the situation that would be faced by a computer engineer who opened up any modern laptop only to find a bewildering array of linked components, as noted earlier with respect to Jonas and Kording's (2017) findings. More generally, computer science has developed many methods for "compiling" grammars into parsers, and the effects of varying these with respect to behavioral and brain measures has generally not been taken fully into account—we simply don't know the "space" of possibilities. In this sense, this result carries the same message as that of Prinz, Bucher & Marder 2004, and also draws attention to Hale et al.'s (2022) adoption of a very particular parser rather than the very different one described below, one that might lead to different conclusions.

How can we deduce what such "wiring diagrams" like that in the table in Figure 1 actually do? It is not easy, and the remainder of this short essay will describe how it is done, and how this "obscures" the grammar a linguist might use. In fact, the diagram in Figure 1 is produced algorithmically from "compiling" a "toy" phrase structure grammar of the kind familiar to linguists—though certainly not of the proper explanatory sort given today's Minimalist theory, or any other linguistic theory. (The same idea scales up, however to a full-scale grammar.) It is designed to demonstrate how a linguistic grammar can be "transformed" into a more opaque internal form that might not even be easily discovered from its "wiring diagram". The (overly) simple grammar is shown in a more recognizable format directly below.

Neurons → Input <i>i</i>	0	1	2	3	4	5	6	7	8	9	10	11	12	13
a	2							2						
b	3		12					3						
c	4							10						
d					7									
e					8									
f					6				6		6			
g					5				9		5			
h	1													
i		13												

Figure 1: A hypothetical table representation of the interneuronal connections of a hypothesized brain region that computes syntax, as revealed by advanced optogenetics at some imaginary future point in time. At the top, each neuron (or small neuronal assembly) is numbered 0 through 13. Inputs to neurons are at the leftmost column, labeled *a* through *i*. The numbers in the cells, e.g., “2” in the top, leftmost cell, indicate that given an input *a*, Neuron 0 activates Neuron 2, and so the system changes state. (Note that some interconnections are empty.)

(Toy) phrase structure grammar that is implemented by the table in Figure 1:

- (1) $S \rightarrow NP VP$
 $VP \rightarrow Verb NP$
 $VP \rightarrow VP PP$
 $NP \rightarrow Determiner Noun$
 $NP \rightarrow NP PP$
 $NP \rightarrow Noun$
 $PP \rightarrow Prep NP$
 $Verb \rightarrow \text{shot, ate, ...}$
 $Prep \rightarrow \text{on, in, ...}$
 $Noun \rightarrow \text{I, elephant, ...}$
 $Determiner \rightarrow \text{a, the, ...}$

In fact, this simple grammar is faithfully replicated by Figure 1's table. Let's see how this works—how this “transparent” grammar in (1), familiar to linguists, is compiled into the table form depicted in Figure 1, so demonstrating the challenge of “Moro's problem.” We will do this in two steps, for ease of understanding. In the first step, we will convert the grammar into an equivalent form, called an (augmented) transition network (ATN) grammar (Woods 1970). In the second step, we show how to turn that network into a more compact table form, as is done in computer science for so-called LR-parsing (Knuth 1965; the acronym “LR” packs together in typical Knuth-like fashion the fact that this is a so-called *Leftmost derivation* done in *Reverse*, as well as that the parsing itself proceeds strictly Left to Right).

First, let's map from the phrase structure grammar above to a transition network as shown in Figure 2. We then recall the way that ATNs are described in order to explain how such a network represents the original grammar.

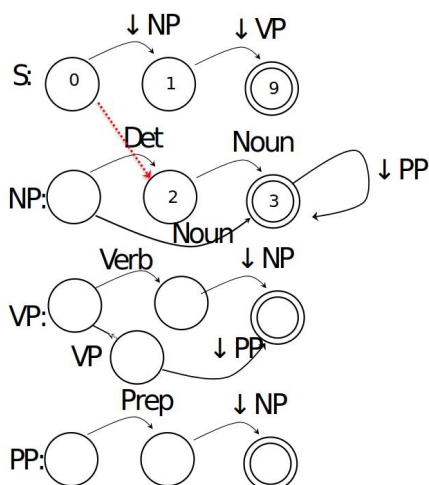


Figure 2: A transition network diagram emulating the phrase structure grammar given above in (1)³

3. An ATN is a set of directed, labeled graphs. Nodes in the graph are circles; arcs

Here is how the transition machine works. As is familiar, the ordering of the nodes and arcs corresponds quite closely to the original grammar. For example, the $S \rightarrow NP VP$ rule has a corresponding S network, that has nodes connected by the arcs labeled NP and VP, and then ends in a final, double-circle state. We can specify what state of the network one is in during the analysis of a sentence by following a “progress dot,” denoted $\bullet$, indicating the point of analysis in terms of the states of the network the system could be in. For example, if we have a sentence such as, “Andrea likes ice-cream” then, before we have processed any words at all, the system is in state $[\bullet S]$, meaning that we are about to start a Sentence.

This representation, with the rule and an indicator of how far the system has made it through that rule, given the input, is called *dotted rule notation*. In addition, since the next arc of the S network is labeled $\downarrow NP$ (looking or seeking for an NP), this also means that we could be in the states $[NP \bullet \text{Det } N]$ and $[NP \bullet \text{Noun}]$ (the two ways that might succeed in finding an NP). If the first word in the sentence is in fact “Andrea,” a Noun, the network can move to the end of the NP network, state 3. Putting all this together, we have the following operation in terms of states and network actions: (Noun, state 0) state 3. Note that if the system reaches state 3, a final state for the NP subnetwork, it has found a complete NP and should go back to the subnetwork (here, the S) that was attempting to confirm that an NP

are labeled with either preterminal (part of speech) or nonterminal names in the corresponding grammar. Phrases like S or NP are represented by separate subgraphs. Phrase names followed by colons like S: also name the subgraphs, corresponding to the left and then the right-hand side of each phrase structure rule. For example, if there is a Noun at some point in the input, then a transition is made from the state (circle) before the Noun to the state after the Noun. If the labeled arc is a phrase name, then that is preceded by a downwards arrow, indicating that the system must seek a phrase of that type and traverse its corresponding subnetwork first. Double circles denote completed phrases or end-states of a phrase. In state 0, the transition calls for a $\downarrow NP$, which means the system moves to the NP subnetwork (like a subroutine call). Given a sentence and a network, we start at the “S:” position, in state 0, and, given the words in the sentence, we attempt to traverse the network to the very end of the sentence, consuming all the input words. The explanation for the addition of the dotted line to the graph is provided in the main text.

was present.

Thus, there are two types of actions in this sense: either the system *scans* over a part of speech, or else it *completes* a phrase like an NP, VP, PP, or S. Alternatively, if the input sentence was “the...”, with a Determiner at the beginning, then the system would have (Det, state 2) state 3. (This is explanation for the immediate jump from state 0 to state 3 shown in Figure 2.)

Now comes the reveal. Looking back to Figure 1, we can see that what we have just described is in fact part of the previously mysterious matrix of input and neuron to neuron signaling: the item *a* in row 1 is now seen to correspond to a “Determiner” (transmitted from some other part of the language system), and the cell below Neuron 0 contains the number 2 in it, the place the system goes to, aligning with the transition we just described for a Determiner. Similarly, *b* corresponds to the input of a Noun, and, at the Start state 0, the matrix directs the system to signal Neuron (state) 3. In this way, the grammar-as-transition network can be seen to be represented as a table of signaling interconnections.

To be sure, we have given only a partial accounting of the details of the full table in Figure 1. For instance, note that a Det(erminer) (row *a*), can trigger column 0 or column 7 to move to state 2. This is because a Det can be found to start an NP as either the object of a VP or a PP—both possibilities are collapsed into state 7. In fact, the way the table is actually built is according to the algorithm developed by Knuth 1965, for what Knuth dubbed “LR parsing.” The *full* table shows that the “states” are really simply collections of the dotted rules described earlier. While there is a lack of space here to explain how this can be done algorithmically, the basic idea is that the “jumps” from e.g., state 0 to state 2 given a Det(erminer) can be collapsed together into equivalence classes: a Det(erminer) jump also occurs if an NP is an Object of the VP or a PP. When we add in these new transitions and collapse similar transitions like the Object NP of a VP or PP, we get the network of state nodes and transition arcs pictured in the diagram in Figure 3, which is essentially that of our original Figure 1 (with dotted rule entries inside the nodes; we have also added an end of sentence marker \$ that is processed, which adds a state to

the system).

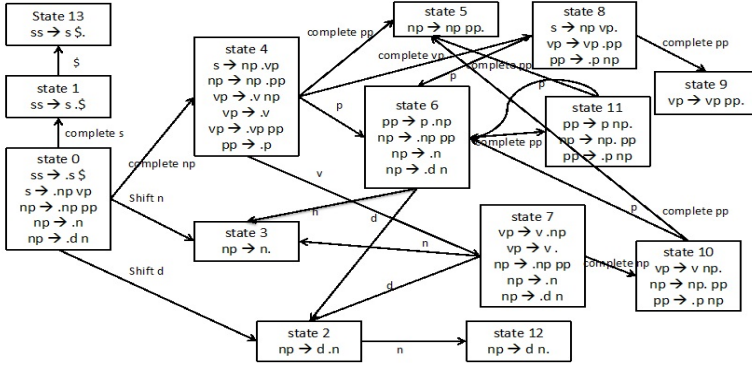


Figure 3: Full LR state and transition diagram corresponding to the phrase structure grammar of (1). States correspond to network states in Figure 1 (and in part to Figure 2); they contain dotted rules denoting possible parsing status points after reading some word sequence. Transitions between states are either *scan* given some part of speech (terminal) in the input/grammar, or *complete* (*phrase*) indicating that a whole phrase of some type (S, NP, VP, PP) has been found at that point. This diagram thus gives a full accounting of what “lies behind” Figure 1’s interconnections.

We have now revealed in full what was behind the neuronal interconnection diagram that was observed, but not understood, by advanced optogenetic methods. The lesson from our short journey into parser construction is that if we knew what the true neuronal signaling interconnections were, unless we supply some outside understanding of *how* such a system was designed, or its function, we might not get very far in our understanding of how a grammar has been encoded in the brain. (Again, this is the same moral also drawn by Jonas & Kording 2017.) In this case, we assumed that while the language acquisition system (including Universal Grammar) had successfully “learned” the right grammar, it was then transformed by some internalized algorithm—compiled—into a different but more efficient form for language use. What we have shown is, of course, just one possible way of “compiling” a grammar out of many possibilities—we

do not know the true grammar or the true compiler, both empirical questions, and so this question becomes a field for inquiry, one that includes how to specify the “space” of possible compilations. Or perhaps the grammar is not compiled at all. Whether compiled or not, we should always be aware that Nature may have hidden the true internal grammar from us in a much more subtle way than might have been imagined, so lending great importance to the answer to Moro’s Problem and others who study it.

Acknowledgements

This brief analysis is dedicated to the inspiration and intellectual and spiritual support that my friend Andrea Moro has provided to me over many decades. Without him, there would be far less to know about the answer to “Moro’s Problem.” The author is also indebted to two reviewers and the Editors for extremely helpful suggestions that greatly improved the presentation; all remaining errors remain mine.

References

- Boyden, E. S., F. Zhang, E. Bamberg, G. Nagel & K. Deisseroth. 2005. Millisecond-timescale, genetically targeted optical control of neural activity. *Nature Neuroscience* 8. 1263–1268.
- Hale, J.T., L. Campanelli, J. Li, S. Bhattasali, C. Pallier & J.R. Brennan. 2022. Neurocomputational models of language processing. *Annual Review of Linguistics* 8. 427–446.
- Jonas, E. & K.P. Kording. 2017. Could a neuroscientist understand a microprocessor? *PLoS Computational Biology* 13(1). e1005268.
- Knuth, D.E. 1965. On the translation of languages from left to right. *Information and Control* 8. 6, 607–639.
- Marr, D. 1982. *Vision: A computational approach*. San Francisco: WH Freeman.
- Moro, A. 2015. *The boundaries of Babel*. Cambridge, MA: MIT Press.
- Moro, A. 2016. *Impossible languages*. Cambridge, MA: MIT Press.
- Musso, M., A. Moro, V. Glauche, M. Rinjtes, J. Reichenbach, C. Büchel & C. Weiller. 2003. Broca's area and the language instinct. *Nature Neuroscience* 6. 7, 774–781.
- Poeppel, D. 2012. The maps problem and the mapping problem: two challenges for a cognitive neuroscience of speech and language. *Cognitive Neuropsychology* 29. 1–2, 34–55.
- Prinz, A.A., D. Bucher & E. Marder. 2004. Similar network activity from disparate circuit parameters. *Nature Neuroscience* 7(12). 1345–1352.
- Riskin, J. 2003. The defecating duck, or, the ambiguous origins of artificial life. *Critical Inquiry* 29. 4, 599–633.
- Woods, W.A. 1970. Transition network grammars for natural language analysis. *Communications of the ACM* 13. 10, 591–606.